AI_Lab01

Name: Nguyen Chi Bao

Student ID: ITCSIU24034

PART 1: LAB REPORT (Answers for Task 3)

1. Data Storage (Internal State)

- To ensure the agent can effectively operate in both empty environments and ones with obstacles—without falling into infinite loops. I equipped it with memory. I implemented a class called MyAgentState to store and manage this information.
- **The Map (self.world):** A 2D array serves as the agent's "brain," storing the status of every grid cell it encounters. Each cell can be marked as UNKNOWN (not yet visited), WALL (an obstacle), CLEAR (safe to move through), DIRT, or HOME (the starting position at coordinates 1,1).
- **Agent's Status:** The agent maintains its current absolute position using coordinates *(pos_x, pos_y)* and also tracks the direction it is facing, which can be NORTH, EAST, SOUTH, or WEST.
- **Last Action:** The agent stores its previous action (last_action), which is important for updating its position. It only changes (pos_x, pos_y) when the last action was FORWARD and no collision occurred.

2. The Main Decision Loop

The main decision-making process is handled in the execute (self, percept) function. This function runs in a loop, guiding how the agent processes information and decides what to do at each step.

1. **Blind Phase**: As required by the lab, the agent spends its first 10 steps moving randomly to simulate being placed in an unknown location. Even though its movement is random, it still keeps track of its coordinates to avoid getting lost.
2. **Reading Sensors & Updating the Map**: After the random phase, the agent begins using its internal memory. It reads inputs from the bump, dirt, and home sensors.
    - If it bumps into something, it determines the cell directly in front and marks it as a WALL in its map.
    - If it detects dirt, it marks the current cell as DIRT; otherwise, it marks it as CLEAR.

3. **Immediate Action**: If the agent is on a dirty tile, it skips all further processing and instantly returns ACTION_SUCK.
4. **Pathfinding (BFS):** If the current tile is clean, the agent applies the Breadth-First Search (BFS) algorithm on its internal map (self.world) to locate the nearest UNKNOWN cell using the shortest path.
5. **Movement**: Once a target is identified, the agent determines how to reach it based on its current direction, then returns the appropriate action: ACTION_TURN_LEFT, ACTION_TURN_RIGHT, or ACTION_FORWARD.

3. How the Agent Handles Task 1 and Task 2

- **Task 1 (No Obstacles):** The agent works perfectly in an empty environment. With no walls to avoid, the BFS algorithm continuously finds the nearest unvisited tile, resulting in a smooth sweeping pattern until the entire area is explored.
- **Task 2 (Handling Obstacles):** While simple reactive agents may get stuck, this model-based agent handles obstacles effectively. Whenever it encounters a bump, it records that position as a WALL in its map. The BFS algorithm then avoids these wall cells, naturally finding alternative paths around obstacles.
- **Returning Home & Shutting Down:** The agent knows it's finished when BFS can no longer find any reachable UNKNOWN tiles, meaning the entire environment has been explored and cleaned. At that point, it sets its goal to *(1, 1)* (Home). BFS computes the shortest path back, and once the agent arrives, it sets `self.alive = False` and returns ACTION_NOP to safely shut down.

4. Design Choices and Alternatives

The implementation of Breadth-First Search (BFS) was selected as the primary navigational strategy to optimize agent efficiency within the simulated environment. In an unweighted grid where every movement and rotation incurs a uniform performance penalty of -1 point, BFS is mathematically guaranteed to identify the shortest path to a target cell. By minimizing the total number of steps, the agent preserves its initial score and maximizes the rewards gained from cleaning dirt and returning home.

The following alternatives were evaluated during the design phase:

- **Depth-First Search (DFS):** While DFS is noted for its lower memory requirements, it lacks optimality in terms of path length. In this specific lab context, the tendency of DFS to produce elongated, non-optimal paths would result in excessive "step" actions, thereby significantly reducing the agent's final performance metric.
- **A-Star (A) Search:*** Although A* is a highly efficient heuristic-based algorithm often used in complex search spaces, its application was deemed unnecessary for this task. Because the vacuum world contains uniform traversal costs rather than weighted terrain, BFS achieves identical path optimality without the additional computational overhead of defining and calculating heuristic functions

Gitlink: https://github.com/accban2006/ai-lab